

Stacks, Queues & the Public/Private Divide

Review worksheet — ADTs, encapsulation, and access control

Name _____ Date _____ Section _____

array[0] Fill in the Blank

one word / token per blank

1. A `class` defaults its members to _____ access; a `struct` defaults its members to _____ access.
2. The keyword that requests memory from the heap instead of the stack is _____.
3. In an array-based stack, the integer that stores the index of the most recently pushed value is conventionally named _____.
4. Adding a value to a stack is called a _____; removing the top value is called a _____.
5. In a linked-list based queue, new nodes are attached at the _____ and removed from the _____.
6. _____ is the principle of bundling data with the methods that act on it, and restricting outside code from touching that data directly.
7. _____ is the principle that lets you call `push()` on a stack without knowing (or caring) whether it's backed by an array or a linked list.
8. A member marked _____ can be called only by code that lives inside the class itself.
9. Fill in the missing keyword so `main()` is allowed to call `push()` :

```
class Stack {
    int *array;
    int top;
    _____:
    void push(int val);
};
```

Word bank public private class struct new encapsulation abstraction top
front rear push pop constructor

array[1] Find the Error

continued on back

CASE A

COMPILES? Y / N

```
class Stack {
    int *array;
    int top;

    Stack(int size) {
        array = new int[size];
        top = -1;
    }

public:
    void push(int val) { array[++top] = val; }
};

int main() {
    Stack S(10);
    S.push(7);
}
```

Circle one: **compiles** / **does not compile**. Which line fails, and why?

Stacks, Queues & the Public/Private Divide

Review worksheet — ADTs, encapsulation, and access control

array[1] Find the Error

2 of 2

CASE B

COMPILES, BUT...

```
class Stack {
public:
    int *array;
    int top;

    Stack(int size) {
        array = new int[size];
    }

    void push(int val) {
        array[++top] = val;
    }
};
```

This compiles and runs. What value does `top` start with, and why does that make `push()` unreliable?

CASE C

COMPILES? Y / N

```
class Queue {
    Node *front;
    Node *rear;
public:
    Queue() { front = rear = nullptr; }
    void push(int val);
};

int main() {
    Queue Q;
    Q.front = nullptr; // line 11
}
```

Name the exact error on line 11, and explain why access control blocks it.

CASE D

COMPILES? Y / N

```
class Node {
    int data;
    Node *next;
public:
    Node(int v)
        : data(v), next(nullptr) {}
}

int main() {
    Node n(5);
}
```

This is a plain syntax error, not an access error. What's missing, and exactly where?

array[2] Short Answer

full sentences

1. Distinguish *encapsulation* from *abstraction* in your own words. Give one example of each from the Stack or Queue class we built in lecture.
2. Why does a linked-list queue need to track both `front` and `rear`, while an array-based stack only needs a single index, `top`?
3. Suppose `array` and `top` were `public` instead of `private` in the Stack class. Write one line of code a careless caller in `main()` could use to corrupt the stack's invariant.
4. Look back at **Case A**. Give one way to fix it so `main()` can still create a `Stack` object.
5. Define LIFO and FIFO in one sentence each, and say which describes the Stack and which describes the Queue.